Runtime state, checkpoints, and provenance

ar089 · 14 August 2026 · Draft

Contents

Developer guide

Runtime state

Runtime state contains the dynamic values required to continue one trajectory: voltages, refractory counters, conductances, projection and input histories, and completed step count. It excludes static parameter values.

A runtime-state signature covers timestep, populations, input shapes, projections, delays, synapses, and state layout. Incompatible state is rejected rather than partially loaded. This supports burn-in followed by a compatible intervention without restarting the circuit.

Runtime-state save, load, validation, and continuation are implemented.

Checkpoints

Parameter checkpoints and runtime state serve different purposes. A parameter checkpoint describes learned or initialized weights. Runtime state describes where one simulation trajectory currently is. A graph-training checkpoint additionally carries named optimizer state, the completed-update count, CPU stochastic state, the resolved execution protocol, initializer metadata, and graph and training digests.

The graph trainer can write both the final update and the lowest-loss update selected during an invocation. Resume validates the complete named parameter set, shapes, dtypes, recipe identity, protocol, and initializer provenance before restoring parameters, AdamW state, the random-number stream, and the next dataset epoch and batch. A partial or positional mapping is rejected.

Provenance

A run should retain the bundle digest, resolved execution protocol, seeds, backend and version, realized initializer statistics, checkpoint identity, and produced artifacts. Dense-array graph runs now emit a versioned execution protocol in `metrics.json`, including source digests, resolved input contracts, dataset metadata, timing, masks, and execution seed. A

resumed run must preserve optimizer state, data order, and stochastic streams when exact continuation is claimed.

The bundle is network provenance, not the complete experiment record. Experiment runners still own conditions, orchestration, analysis, figures, and publication.

API reference

`GraphRuntimeState`

```
GraphRuntimeState(
    signature,
    compatibility,
    completed_steps,
    voltages,
    refractory,
    conductances,
    population_histories,
    input_histories,
)
```

Every tensor group is keyed by stable graph identifier. `.detached(device="cpu")` returns a cloned, detached state on the requested device. Static parameters are not included.

Save, load, and compatibility

```
save_runtime_state(path, state) -> Path
load_runtime_state(path, *, device="cpu") -> GraphRuntimeState
runtime_state_compatibility(plan) -> dict
runtime_state_signature(plan) -> str
```

Runtime state uses schema `tools/snn.graph-runtime-state/v1`. Loading verifies the manifest and tensor payload. Simulation compares the stored signature and compatibility structure with the current graph plan before restoring any state.

Resume execution

```
result = simulate(spec, runtime_state=previous.runtime_state)
```

A resume request must preserve graph structure, timestep, population sizes, projections, delays, synapses, parameter shapes, batch shape, and state dtypes.
`result.runtime_state.completed_steps` reports the cumulative trajectory length.

Training checkpoints

```
save_training_checkpoint(path, checkpoint) -> Path
load_training_checkpoint(path, *, device="cpu") -> TrainingCheckpoint
legacy_parameter_map_v1(graph) -> dict[str, str]
import_legacy_parameters_v1(graph, state_dict) -> ParameterInterchange
export_legacy_parameters_v1(graph, parameters) -> ParameterInterchange
```

Training checkpoints use schema `tools/snn.training-checkpoint/v1` and store a JSON manifest beside a digest-verified compressed tensor payload. Parameters and optimizer tensors are keyed by stable graph parameter id. The manifest authenticates the graph and training recipes, completed update, resolved execution protocol, realized initialization metadata, optimizer scalars, selected loss, tensor layout, and dataset iterator position.

`ExecutionSpec.checkpoint` resumes graph training from a checkpoint directory. The `save_final_checkpoint` and `save_selected_checkpoint` options persist the final and invocation-selected states. `legacy_parameter_map_v1` provides the explicit one-layer legacy adapter map and fails closed for graphs that cannot be represented without omissions or ambiguity.

Simulation and inference requests also accept a graph training-checkpoint directory. They authenticate the payload and graph digest, require exact parameter names, shapes, and dtypes, and load parameters without restoring optimizer, random-stream, or data-order state. Inference metrics retain the checkpoint format, path, graph and training digests, completed update, and selected loss. A non-directory checkpoint remains the explicit legacy PyTorch state-file route.

Parameter interchange uses schema `tools/snn.legacy-parameter-interchange/v1`. Import and export require the exact six supported one-layer keys, runtime shapes, floating dtypes, and complete graph coverage. The returned provenance records direction, mapping version, and the full semantic map. This is parameter-only interchange: legacy optimizer objects are not relabelled as portable graph-training checkpoints.

Provenance fields

Graph simulation metrics include device, recording profile, build and simulation timing, runtime-state schema and signature, completed steps, and the resolved execution protocol. Dense file bindings add source paths, SHA-256 digests, array keys, shapes, dtypes, dataset metadata, timing, masks, and execution seed.

Next: Compatibility, status, and extension